

BinaryWriter and BinaryReader in C# with Examples

In this article, I am going to discuss **BinaryWriter and BinaryReader in C#** with Examples. Please read our previous article where we discussed [TextWriter and TextReader Classes in C#](#) with Examples. At the end of this article, you will understand what BinaryWriter and BinaryReader are in C# and when and how to use BinaryWriter and BinaryReader in C# with Examples.

What is BinaryWriter Class in C#?

The BinaryWriter class in C# is used to write Primitive type data types such as int, uint, or char in the form of binary data to a stream. It is present under the System.IO namespace. As its name says BinaryWriter writes binary files that use a specific data layout for its bytes.

1. The BinaryWriter in C# creates a binary file that is not human-understandable but the machine can understand it.
2. It supports writing strings in a specific encoding.
3. In order to create an object of BinaryWriter, we need to pass an object of Stream to the constructor of the BinaryWriter class.
4. The BinaryWriter class in C# provides methods that simplify writing primitive data types to a stream.
5. If you don't provide types of encoding while creating objects then default encoding UTF-8 will be used.

If you go to the definition of BinaryWriter class, then you will see the following structure.

```

namespace System.IO
{
    ...public class BinaryWriter : IDisposable
    {
        ...public static readonly BinaryWriter Null;
        ...protected Stream OutStream;

        ...public BinaryWriter(Stream output);
        ...public BinaryWriter(Stream output, Encoding encoding);
        ...public BinaryWriter(Stream output, Encoding encoding, bool leaveOpen);
        ...protected BinaryWriter();

        ...public virtual Stream BaseStream { get; }

        ...public virtual void Close();
        ...public void Dispose();
        ...public virtual void Flush();
        ...public virtual long Seek(int offset, SeekOrigin origin);
        ...public virtual void Write(char[] chars, int index, int count);
        ...public virtual void Write(string value);
        ...public virtual void Write(float value);
        ...public virtual void Write(ulong value);
        ...public virtual void Write(long value);
        ...public virtual void Write(uint value);
        ...public virtual void Write(int value);
        ...public virtual void Write(ushort value);
        ...public virtual void Write(short value);
        ...public virtual void Write(byte[] buffer, int index, int count);
        ...public virtual void Write(double value);
        ...public virtual void Write(char[] chars);
        ...public virtual void Write(char ch);
        ...public virtual void Write(byte[] buffer);
        ...public virtual void Write(sbyte value);
        ...public virtual void Write(byte value);
        ...public virtual void Write(bool value);
        ...public virtual void Write(decimal value);
        ...protected virtual void Dispose(bool disposing);
        ...protected void Write7BitEncodedInt(int value);
    }
}

```

Methods of BinaryWriter Class in C#:

1. **Write(String):** This method is used to write a length-prefixed string to this stream in the current encoding of the BinaryWriter and advances the current position of the stream in accordance with the encoding used and the specific characters being written to the stream.
2. **Write(float):** This method is used to write a four-byte floating-point value to the current stream and advances the stream position by four bytes.

3. **Write(long)**: This method is used to write an eight-byte signed integer to the current stream and advances the stream position by eight bytes.
4. **Write(Boolean)**: This method is used to write the one-byte Boolean value to the present stream; 0 represents false while 1 represents true.
5. **Write(Byte)**: This method is used to write an unsigned byte to the present stream and then it advances the position of the stream by one byte.
6. **Write(Char)**: This method is used to write Unicode characters to the present stream and also it advances the present stream position according to the character encoding used and according to the characters being written to the present stream.
7. **Write(Decimal)**: This method is used to write a decimal value to the present stream and also it advances the position of the current stream by sixteen bytes.
8. **Write(Double)**: This method is used to write an eight-byte floating-point value to the present stream and then it also advances the position of the current stream by eight bytes.
9. **Write(Int32)**: This method is used to write a four-byte signed integer to the present stream and then it advances the position of the current stream by four bytes.

How to Create an Instance of BinaryWriter Class in C#?

There are four overloaded constructors available in BinaryWriter class to create a BinaryWriter instance. They are as follows:

1. **public BinaryWriter(Stream output)**: It initializes a new instance of the BinaryWriter class based on the specified stream and uses UTF-8 encoding. Here, the parameter output specifies the output stream.
2. **public BinaryWriter(Stream output, Encoding encoding)**: It initializes a new instance of the BinaryWriter class based on the specified stream and character encoding. Here, the parameter output specifies the output stream and the parameter encoding specifies the character encoding to use.
3. **public BinaryWriter(Stream output, Encoding encoding, bool leaveOpen)**: It initializes a new instance of the BinaryWriter class based on the specified stream and character encoding, and optionally leaves the stream open. Here, the parameter output specifies the output stream and the parameter encoding specifies the character encoding to use and the parameter leaveOpen specifies true to leave the stream open after the BinaryWriter object is disposed otherwise, false.
4. **protected BinaryWriter()**: It initializes a new instance of the System.IO.BinaryWriter class that writes to a stream.

We can create an object of BinaryWriter class inside the using block so that the memory occupied by the object will be released automatically when the work of the object is completed and it is no longer needed. The syntax is given below.

```
using (BinaryWriter binaryWriter = new BinaryWriter(File.Open(fileName, FileMode.Create )))  
{  
    //user code  
}
```

Here is, File.Open() method returns the FileStream object which helps to create an instance of BinaryWriter.

How BinaryWriter Works in C#?

In C#, the BinaryWriter class is used to write binary data to a file or we can say that it is used to create binary files. It helps us write primitive data types such as int, char, double, etc. in binary format to a stream. It also helps us write strings in a particular character encoding format.

In order to work with BinaryWriter in C#, first, we need to import the System.IO namespace. Then, we need to create an instance of the BinaryWriter class by using the new operator and bypassing a Stream object to the constructor of BinaryWriter. There are multiple versions of Constructors available in BinaryWriter class. You can use any one of them.

To create an instance of BinaryWriter in C#, we generally provide a Stream object to its constructor at the same time we can also provide optional parameters that specify the encoding to be used while writing the file. In case, the user does not provide any character encoding then UTF-8 encoding will be used by default.

The BinaryWriter class in C# provides different Write() methods for different types of data. These methods are used to write data to the binary file.

Example to Understand BinaryWriter Class in C#

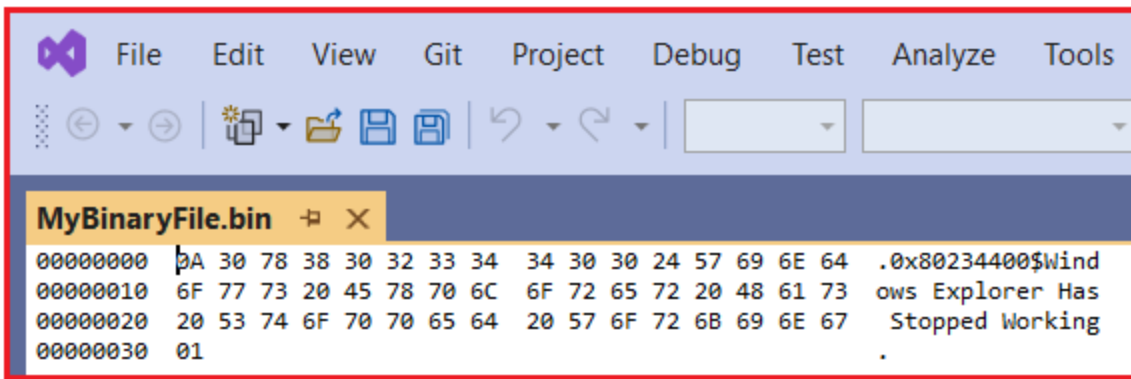
In the below example, we create a new binary file at location "D:\MyBinaryFile.bin" and then store error log information in it.

```
using System;
using System.IO;
namespace FileHandlingDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            using (BinaryWriter writer = new
BinaryWriter(File.Open("D:\\MyBinaryFile.bin", FileMode.Create)))
            {
                //Writting Error Log
                writer.Write("0x80234400");
                writer.Write("Windows Explorer Has Stopped Working");
                writer.Write(true);
            }
            Console.WriteLine("Binary File Created!");
            Console.ReadKey();
        }
    }
}
```

When you run the above code, you will get the following output.

A screenshot of a console window with a black background and white text. The text "Binary File Created!" is displayed in a monospaced font.

Now, in the D drive, the MyBinaryFile.bin file should be created and if you open this file with visual studio, then you will see the following.



So, when you open the file D:\ MyBinaryFile.bin in the visual studio the file may look like the above. It is hard to understand but it is a more efficient and machine-level representation of data. This is because the data is not encoded in the text file. Don't worry when you read your data using BinaryReader Class you will get the exact data that you saved.

Note: The main advantages of Binary information are that it is not easily human-readable and storing files as Binary format is the best practice for space utilization.

What is BinaryReader class in C#?

If you have a binary file (with .bin extension) stored in your machine and if you want to read the binary data then you need to use the BinaryReader class in C#. That means the BinaryReader class in C# is used to reading binary file data. A binary file stores data in a different layout that is more efficient for machines but not convenient for humans. BinaryReader makes this job simpler and shows you the exact data stored in the binary file.

The BinaryReader class belongs to the System.IO namespace. BinaryReader is used to read primitive data types as binary values in a particular encoding stream. BinaryReader works with Stream object i.e. in order to create an object of BinaryReader, we need to pass the Stream object to its constructor. BinaryReader class has three overloaded constructors to work with binary data. By default, BinaryReader uses UTF-8 encoding to read data until we specify other character encodings while creating its object.

1. The BinaryReader class in C# handles Binary (.bin) files.
2. It reads primitive data types as binary values in a specific encoding.
3. The BinaryReader class provides methods that simplify reading primitive data types from streams.

If you go to the definition of BinaryWriter class, then you will see the following structure.

```

namespace System.IO
{
    ...public class BinaryReader : IDisposable
    {
        ...public BinaryReader(Stream input);
        ...public BinaryReader(Stream input, Encoding encoding);
        ...public BinaryReader(Stream input, Encoding encoding, bool leaveOpen);

        ...public virtual Stream BaseStream { get; }

        ...public virtual void Close();
        ...public void Dispose();
        ...public virtual int PeekChar();
        ...public virtual int Read();
        ...public virtual int Read(byte[] buffer, int index, int count);
        ...public virtual int Read(char[] buffer, int index, int count);
        ...public virtual bool ReadBoolean();
        ...public virtual byte ReadByte();
        ...public virtual byte[] ReadBytes(int count);
        ...public virtual char ReadChar();
        ...public virtual char[] ReadChars(int count);
        ...public virtual decimal ReadDecimal();
        ...public virtual double ReadDouble();
        ...public virtual short ReadInt16();
        ...public virtual int ReadInt32();
        ...public virtual long ReadInt64();
        ...public virtual sbyte ReadSByte();
        ...public virtual float ReadSingle();
        ...public virtual string ReadString();
        ...public virtual ushort ReadUInt16();
        ...public virtual uint ReadUInt32();
        ...public virtual ulong ReadUInt64();
        ...protected virtual void Dispose(bool disposing);
        ...protected virtual void FillBuffer(int numBytes);
        ...protected internal int Read7BitEncodedInt();
    }
}

```

Methods of BinaryReader Class in C#:

The BinaryReader class in C# provides many Read() methods to read different primitive data types from a stream. Such as the ReadString() method of BinaryReader is used to read the next byte as a string value and also it advances the current position in the stream by one byte. The different types of Read() methods of BinaryReader class are as follows:

1. **Read():** It is used to read characters from the underlying stream and advances the current position of the stream in accordance with the Encoding used and the specific character being read from the stream. It returns the next character from the input stream, or -1 if no characters are currently available.
2. **ReadBoolean():** It is used to read a Boolean value from the current stream and advances the current position of the stream by one byte. It returns true if the byte is nonzero; otherwise, false.
3. **ReadByte():** It is used to read the next byte from the current stream and advances the current position of the stream by one byte. It returns the next byte read from the current stream.

4. **ReadChar():** It is used to read the next character from the current stream and advances the current position of the stream in accordance with the Encoding used and the specific character being read from the stream. It returns a character read from the current stream.
5. **ReadDecimal():** It is used to read a decimal value from the current stream and advances the current position of the stream by sixteen bytes. It returns a decimal value read from the current stream.
6. **ReadDouble():** It is used to read an 8-byte floating-point value from the current stream and advances the current position of the stream by eight bytes. It returns an 8-byte floating-point value read from the current stream.
7. **ReadInt32():** It is used to read a 4-byte signed integer from the current stream and advances the current position of the stream by four bytes. It returns a 4-byte signed integer read from the current stream.
8. **ReadInt64():** It is used to read an 8-byte signed integer from the current stream and advances the current position of the stream by four bytes. It returns an 8-byte signed integer read from the current stream.
9. **ReadString():** It is used to read a string from the current stream. The string is prefixed with the length, encoded as an integer seven bits at a time. It returns the string being read.

How to Create an Instance of BinaryReader Class in C#?

There are three overloaded versions of constructors available in BinaryReader class to create an instance of BinaryReader class. They are as follows:

1. **public BinaryReader(Stream input):** It initializes a new instance of the System.IO.BinaryReader class based on the specified stream and using UTF-8 encoding. Here, the parameter input specifies the input stream.
2. **public BinaryReader(Stream input, Encoding encoding):** It initializes a new instance of the System.IO.BinaryReader class based on the specified stream and character encoding. Here, the parameter input specifies the input stream and the parameter encoding specifies the character encoding to use.
3. **public BinaryReader(Stream input, Encoding encoding, bool leaveOpen):** It initializes a new instance of the System.IO.BinaryReader class based on the specified stream and character encoding, and optionally leaves the stream open. Here, the parameter input specifies the input stream and the parameter encoding specifies the character encoding to use and the parameter leaveOpen specifies true to leave the stream open after the BinaryReader object is disposed otherwise, false.

How BinaryReader work in C#?

The BinaryReader class in C# is used to read binary information i.e. it is used to read data stored in binary files (file with .bin extension). The binary file stores data in a way that can be easily understood by a machine but for human beings, it is very difficult to understand. In order to work with BinaryReader, first, we need to import the System.IO namespace. Then, we need to create an instance of BinaryReader class with the help of the new operator and using any of the available constructors. While creating the instance of BinaryReader class, we need to pass the input stream as a parameter to the constructor.

While creating an instance of BinaryReader we can also optionally specify the character encoding to be used if we do not specify the encoding, by default it will use UTF-8 encoding. Along with this, we can also optionally specify if we want the stream to be opened after the object of BinaryReader is disposed of as shown in the below statement.

```
BinaryReader binary_reader = new BinaryReader(inputStream, encoding, true);
```

Once we created the object of BinaryReader class, then with the help of different Read() methods of BinaryReader class, we can read data from the file.

Example to Understand BinaryReader Class in C#:

In the below example I have created 2 methods WriteDataToBinaryFile() and ReadDataFromBinaryFile(). WriteDataToBinaryFile() method is used to store some information in D:\MyBinaryFile2.bin file and ReadDataFromBinaryFile() method is used to read data from MyBinaryFile2.bin file and displays the data on the console.

```
using System;
using System.IO;
namespace FileHandlingDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            WriteDataToBinaryFile();
            ReadDataFromBinaryFile();
            Console.ReadKey();
        }
        static void WriteDataToBinaryFile()
        {
            using (BinaryWriter writer = new
BinaryWriter(File.Open("D:\\MyBinaryFile2.bin", FileMode.Create)))
            {
                //Writting Error Log
                writer.Write("0x80234400");
                writer.Write("Windows Explorer Has Stopped Working");
                writer.Write(true);
            }
        }
        static void ReadDataFromBinaryFile()
        {
            using (BinaryReader reader = new
BinaryReader(File.Open("D:\\MyBinaryFile2.bin", FileMode.Open)))
            {
                Console.WriteLine("Error Code : " + reader.ReadString());
                Console.WriteLine("Message : " + reader.ReadString());
                Console.WriteLine("Restart Explorer : " + reader.ReadBoolean());
            }
        }
    }
}
```

Output:

```
Error Code : 0x80234400  
Message : Windows Explorer Has Stopped Working  
Restart Explorer : True
```

Note: The BinaryWriter and BinaryReader Classes in C# are used to read and write primitive data types and strings. If you deal only with primitive types, then this is the best stream to use. Remember that this data is not easily readable by a human being as the contents stored in the file are in binary form.

In the next article, I am going to discuss [StringWriter and StringReader in C#](#) with Examples. Here, in this article, I try to explain **BinaryWriter and BinaryReader in C#** with Examples. I hope you enjoy this BinaryWriter and BinaryReader Classes in C# with Examples article. I would like to have your feedback. Please post your feedback, question, or comments about this BinaryWriter and BinaryReader in C# with Examples article.